

L2_switch 设计开发实验说明

主题	FAST_2.0 L2_switch 设计开发说明
文档号	
创建时间	2024-10-18
最后修改	2024-10-23
版本号	1.0
文件名	FAST_2.0 L2_switch 设计开发说明.docx
文件格式	docx

目录

L2_switch 设计开发实验说明	1
一、 L2_switch 简介	3
二、 实验目标	3
三、 L2_switch 整体架构	3
四、 L2_switch 接口定义	4
4.1 iv_xmac/ov_outport	4
4.1.1 信号定义	5
4.1.2 时序定义	5
4.2 Localbus	5
4.2.1 信号定义	6
4.2.2 时序定义	6
4.3 rv_index 信号定义	7
五、 模块介绍	7
5.1 目的 MAC 端口索引 (LID)	7
5.1.1 模块信号定义	8
5.1.2 模块处理流程	9
5.1.3 仿真时序分析	9
5.2 索引端口管理 (MI)	10
5.2.1 模块信号定义	10
5.2.2 模块处理流程	11
5.2.3 仿真时序分析	11
5.3 查表 (LUP)	13
5.3.1 模块信号定义	13
5.3.2 模块处理流程	14
六、 部分代码注释	14
七、 功能验证	17

一、L2_switch 简介

L2_switch(二层交换模块) 是 FAST 架构下硬件中的主要功能模块。该模块是基于 FAST 平台开发的简单教学模块，其目的是让用户通过自己补全目的 mac 端口索引（lookup_index_dmac）模块，索引端口管理（manager_index）模块加深对二层交换 mac 学习的理解，同时提升 fpga 工程实践能力。

本文档对 L2_switch 的设计进行简要说明，包括 L2_switch 的整体实现框架，数据格式和接口的定义，子模块实现功能描述等，帮助用户进一步完成 L2_switch 开发。

二、实验目标

补充 LID、MI 两个模块中的状态机的逻辑，使得该工程能够顺利完成二层自学习交换的功能，并且能够通过第六章中的测试。

三、L2_switch 整体架构

L2_switch 实现架构如下图 1 所示，由 3 个子模块组成，分别为目的 MAC 端口索引模块（LID），索引端口管理模块（MI），查表模块（LUP）。iv_xmac/ov_outport 为输入 mac 输出 mac 查表端口的逻辑处理数据路径,local bus 为上位机访问当前查表模块表项的接口，可以随时访问当前查表模块学习到的 mac 和端口信息。

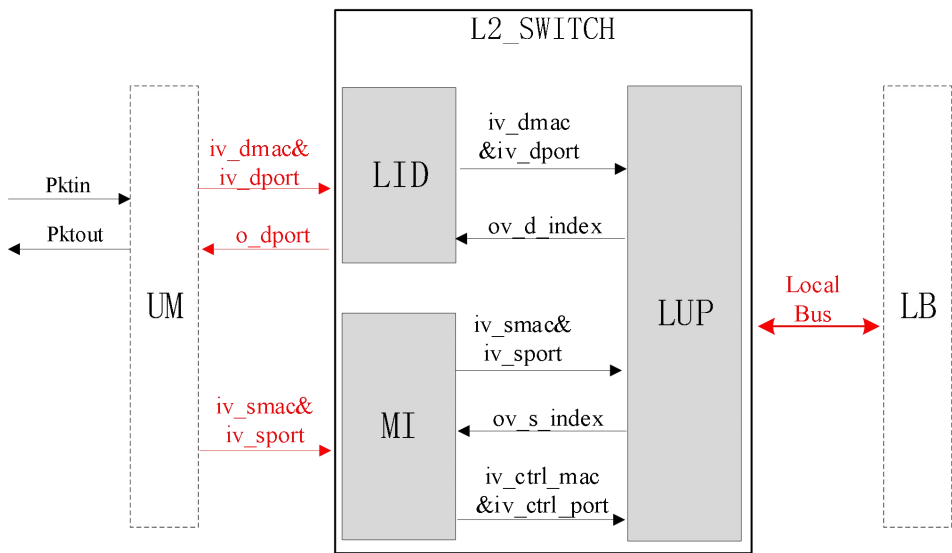


图 1 L2_SWITCH 整体架构

整体架构工作流程如下：

1. 当报文通过千兆网口进入 FAST 平台后，首先由 Pktin 接口进入用户(UM)模块，经用户（UM）模块解析提取出报文的源 mac 地址（iv_smac），源端口（iv_sport），目的 mac 地址（iv_dmac），目的端口（iv_dport）。
2. 提取源 mac 地址（iv_smac）和源端口（iv_sport）进入索引端口管理（MI）

模块，该模块将有效的源 mac 和源端口送入查表（LUP）模块。查表（LUP）模块构建了一个 mac 地址和端口映射转发表，如下表 1 所示。该转发表含有 16 条表项，每条表项存储一个 mac 地址和对应的端口号，当源 mac 地址输入查表(LUP)模块时，执行查表功能，对 16 条表项同时查表匹配，输出源 mac 查表得到的索引值（ov_s_index），该索引值包含在查找表中 16 条查表命中的 mac 地址对应的端口信息。

转发表		
序号	MAC 地址	端口
0	62-FE-F7-11-89-A3	0
...	...	...
15	63-FA-D7-13-80-A4	5

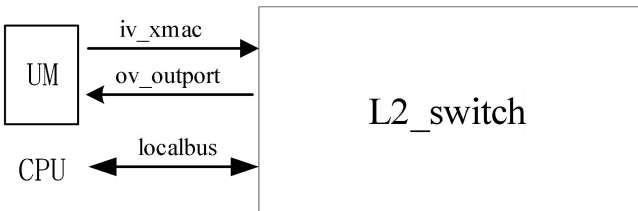
表 1

3. 进行 MAC 地址查找得到的源端口索引（ov_s_index）信息反馈回索引端口管理（MI）模块处理，根据索引信息判断查找表中是否存在源 mac 地址表项，该源 mac 对应的源端口信息是否与当前输入报文的源端口一致，如果不一致，则生成配置 mac（iv_ctrl_mac）和配置端口（iv_ctrl_port）给查表（LUP）模块，对查表（LUP）模块进行转发表项的新建或维护。

4. 其次，经过用户（UM）模块提取的目的 MAC（iv_dmac）输入到目的 MAC 端口索引（LID）模块。当报文是单播报文，该目的 MAC（iv_dmac）输出到查表（LUP）模块，通过转发表查询当前目的 MAC 得到目的转发索引值（ov_d_index），目的转发索引值（ov_d_index）反馈回目的端口索引（LID）模块，进一步提取查询到的转发端口（o_dport）输出给用户（UM）模块。当报文是广播报文，在目的 MAC 端口索引（LID）模块内直接将目的端口号取反，设置端口泛洪。

5. 最后，当前转发表的表项在查表（LUP）模块可以通过管理接口 local_bus 总线传输至上位机打印显示。

四、L2_switch 接口定义



L2_switch 模块定义了 2 种不同类型的接口，分别是数据接口（iv_xmac/ov_outport）、管理接口(Localbus)。

4.1 iv_xmac/ov_outport

数据接口(iv_xmac/ov_outport)是从网络报文中得到的 mac 地址信息和处理得到的输出端口，由于该模块只专注于二层交换 mac 地址的学习及目 mac 的端口查找，故输入端口只有提取后的 mac 信号，不包括完整的报文分组。

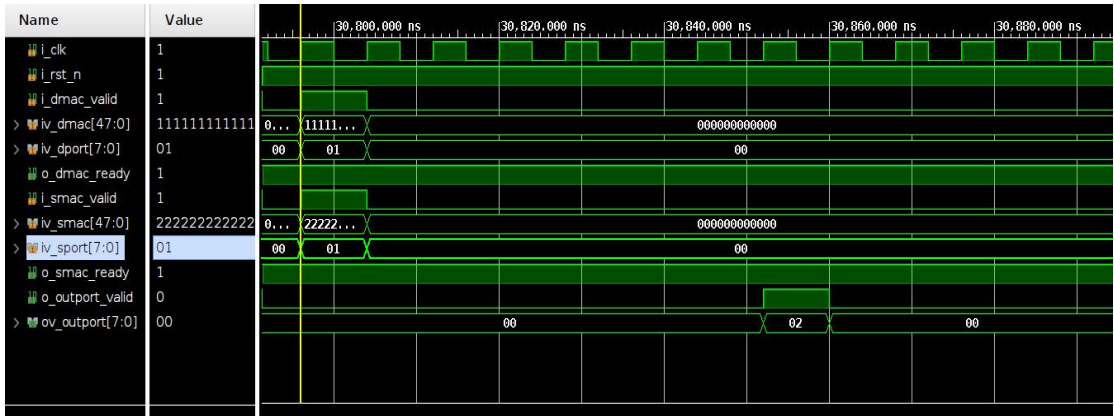
4.1.1 信号定义

接口类型	信号名称	方向	信号描述
xmac	i_xmac_valid	i	L2_switch 接收 mac 有效指示
	iv_xmac[47:0]	i	L2_switch 接收 mac 值
	iv_xport[7:0]	i	L2_switch 接收端口
	o_xmac_ready	o	L2_switch 输出 mac 接收状态指示
outport	o_outport_valid	o	L2_switch 输出端口值有效状态指示
	ov_outport[7:0]	o	L2_switch 输出单端口值 (bitmap) 8'b10000000 端口 7 8'b01000000 端口 6 8'b00100000 端口 5 8'b00010000 端口 4 8'b00001000 端口 3 8'b00000100 端口 2 8'b00000010 端口 1 8'b00000001 端口 0 L2_switch 输出多端口值 8'b00000011 同时输出端口 0、端口 1 8'b01111111 同时输出端口 0~6

4.1.2 时序定义

iv_xmac/ov_outport 接口时序如下图所示，当 o_xmac_ready 信号为高时接口有 mac 数据输入，i_xmac_valid 信号为高时 iv_xmac[47:0] 和 iv_xport[7:0] 数据有效，经过 L2_switch 模块处理，o_outport_valid 信号为高时，输出有效端口信号 ov_outport[7:0]。

下图为 iv_xmac/ov_outport 接口仿真时序，



4.2 Localbus

Localbus 管理接口(localbus)是软件对 L2_switch 进行配置访问操作的接口，接口传输的控制信号由定义软件发送，L2_switch 处理后发送 ack 回

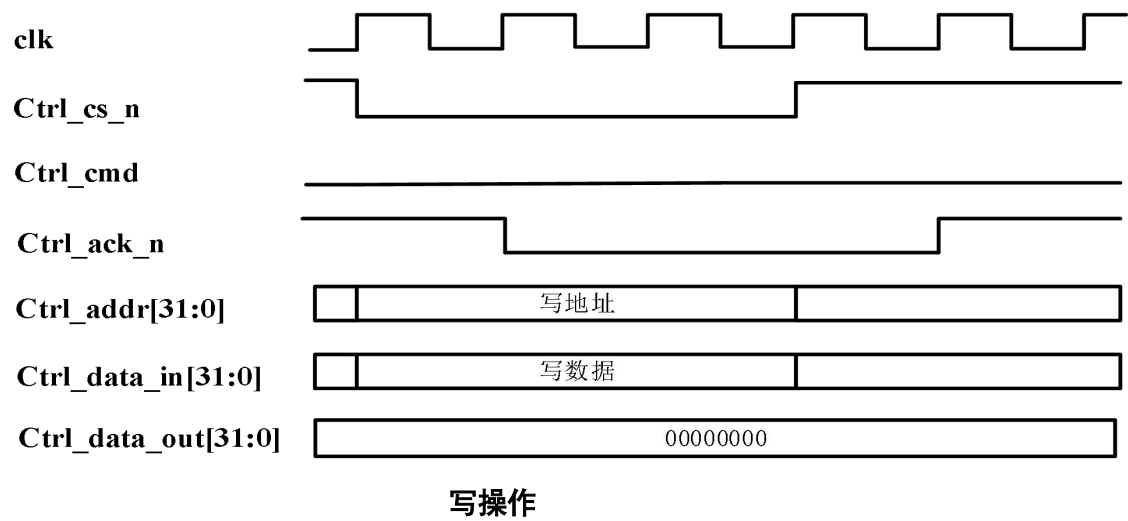
应。Localbus 接口可以用来读写配置硬件逻辑中定义的寄存器或 RAM。功能与 Cin/Cout 相似，二者实现方式不同，用户可以根据自己对这两种接口的理解和具体需求来选择。

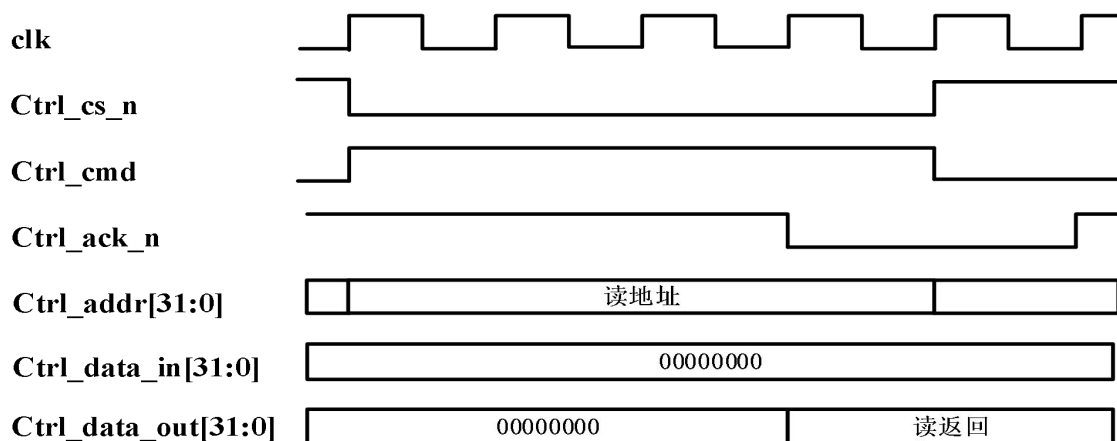
4.2.1 信号定义

接口类型	信号名称	方向	信号描述
Local_bus	Ctrl_cs_n	in	片选信号，低有效
	Ctrl_cmd	in	读写操作信号，低写高读
	Ctrl_addr[15:0]	in	读写操作地址
	Ctrl_data_in[31:0]	in	写数据
	Ctrl_data_out[31:0]	out	读数据
	Ctrl_ack_n	out	操作完成响应，低有效

4.2.2 时序定义

Localbus 接口的时序分为读操作和写操作两种，以 localbus 与用户（um）模块通信为例，时序如下图所示，片选信号 local2um_cs_n 信号为低表示读写操作开始，如果是写操作，则 local2um_wr_rd 信号为低，local2um_addr 信号的值为写地址，local2um_wdata 信号的值为写数据，模块操作完成后，将 um2local_ack_n 置低进行回应，上级模块将 local2um_cs_n 信号信号拉高，一次写操作完成；如果是读操作，local2um_wr_rd 信号为高，local2um_addr 信号的值为读地址，um2local_rdata 信号的值为读返回数据，模块操作完成后，将 um2local_ack_n 置低进行回应，上级模块将 local2um_cs_n 信号信号拉高，一次读操作完成。



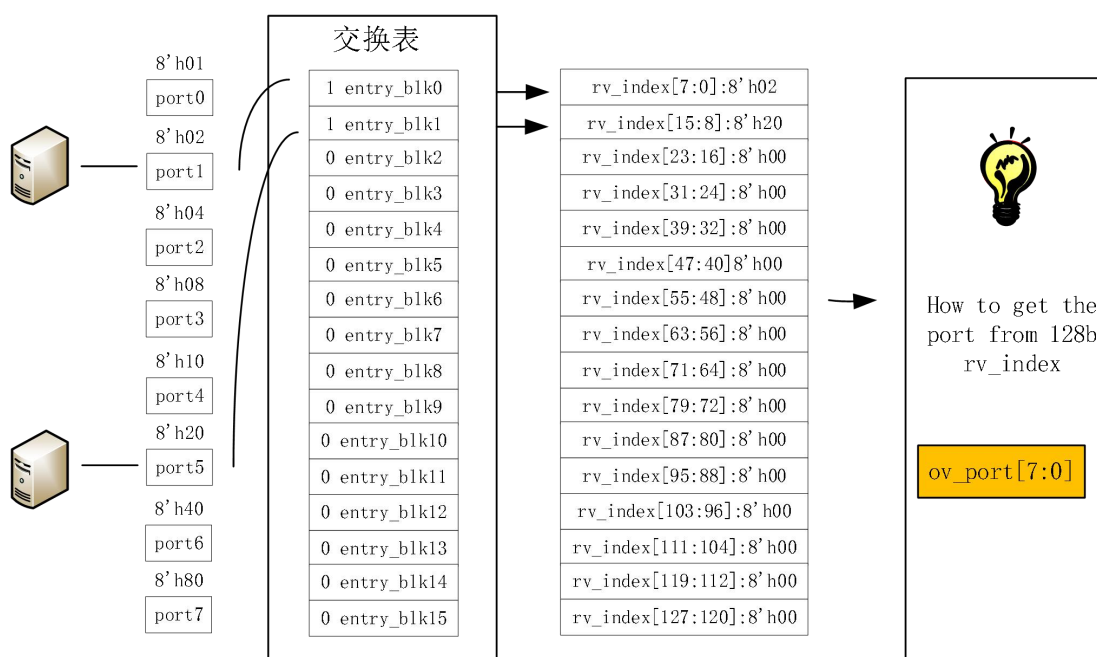


读操作

下图为 Localbus 接口读写仿真时序，



4.3 rv_index 信号定义



当计算机接入对应的端口，将该端口号按顺序送进交换表表项，16 个表项同时记录更新和输出查找表项，总共输出 $16 \times 8 = 128b$ ，最终需要用一定的逻辑从 128b 中得到目的 mac 查找的端口值。

五、模块介绍

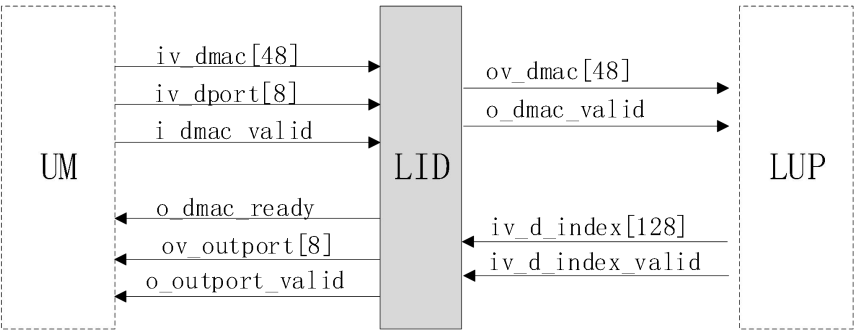
5.1 目的 MAC 端口索引 (LID)

目的 MAC 端口索引模块（LID）实现的功能有：

- 判别单播目的 MAC 和广播目的 MAC，单播目的 MAC 输出送至查表（LUP）模块，广播目的 MAC 直接将转发端口设置为泛洪模式；
- 根据查表（LUP）模块反馈的端口索引值（iv_d_index）得到目的 mac 匹配的转发端口。

5.1.1 模块信号定义

目的 MAC 端口索引（LID）模块接口和信号定义如下图所示，用户（UM）为报文输入模块，从该模块得到目的 mac 和目的端口输入到 LID 模块等待处理，查表（LUP）模块处理有效目的 mac 并进行端口表项查找，用户模块（UM）为后级处理模块，在该模块使用目的 mac 查表后得到的目的转发端口。



LID 模块信号定义表

信号名称	方向	信号描述
i_dmac_valid	i	LID 接收目的 mac 有效指示
iv_dmac[47:0]	i	LID 接收目的 mac 值
iv_dport[7:0]	i	LID 接收目的端口
o_dmac_ready	o	LID 输出 mac 接收状态指示，置 1 时有效
o_dmac_valid	o	LID 输出目的 mac 有效指示信号
o_dmac	o	LID 判别需查询端口号的目的 mac
iv_d_index_valid	i	LID 查询目的 mac 得到端口号有效指示信息
iv_d_index	i	LID 查询目的 mac 得到的端口号信息
o_outputport_valid	o	L2_switch 输出端口值有效状态指示
ov_outputport[7:0]	o	L2_switch 输出单端口值 8'b10000000 端口 7 8'b01000000 端口 6 8'b00100000 端口 5 8'b00010000 端口 4 8'b00001000 端口 3 8'b00000100 端口 2 8'b00000010 端口 1 8'b00000001 端口 0 L2_switch 输出多端口值 8' b00000011 同时输出端口 0、端口 1

	8' b01111111 同时输出端口 0~6
--	-------------------------

5.1.2 模块处理流程

模块复位后，当检测到 i_dmac_valid 为高，将 iv_dmac[47:0]和 iv_dport[7:0]一起写入 fifo 中，当 fifo 不为空的时候，启动状态机 rv_state_s，开始进行对目的 mac (iv_dmac) 和目的端口 (iv_dport) 的处理。

- dmac and dport process : 此逻辑块的功能是处理已缓存到 fifo 中的目的 dmac 和对应端口号，需执行以下功能
 - 1.读出 fifo 内缓存 dmac，根据 dmac 判断是广播 mac 还是单播，将需要进行目的 mac 端口查找的单播 mac 输出。
 - 2.若是广播 dmac，则将输入的目的端口号按位取反输出，若是单播 mac，则将目的 o_dmac 输出到查找 (LUP) 模块，对目的 mac 对应的端口号表项进行查询。
 - 3.等待查找 (LUP) 模块输入有效的目的端口索引号(iv_d_index)。
 - 4.处理查找 (LUP) 模块查询的目的端口索引号，这里因为每个表项储存一个端口号 (8b) 信息，一共设置了 16 条表项，所以一共是 128b，对这 128b 信息进一步处理，得到 8b 的目的 mac 端口信息 (ov_outport)。
 - 5.将最终处理得到的端口信息 (ov_outport) 输出。
- dmac fifo : 此逻辑块的例化了一个 fifo 用于缓存从接收端口得到的目的 mac。

5.1.3 仿真时序分析

仿真模拟以下情况：

对 0 号端口发送 64 字节的 TCP_A 报文，对 1 号端口发送 64 字节的 TCP_B 报文，报文内容分别为：

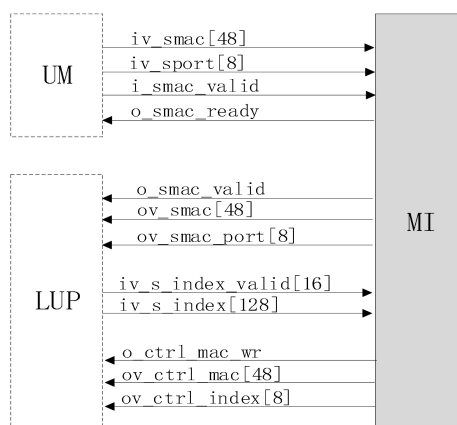
```
TCP_A[511:0] <= {128'h1111111111111222222222222208004500,
                  128'h00320000400040063ac4000000010000,
                  128'h00020000000000000000000000000000,
                  128'h0000ffd8000000000000000000000000};
TCP_B[511:0] <= {128'h222222222222111111111111108004500,
                  128'h00320000400040063ac4000000020000,
                  128'h00010000000000000000000000000000,
                  128'h0000ffd8000000000000000000000000};
```

通过仿真观察 lookup_index_dmac 模块，可以发现：

当输入 TCP_A 报文时，此时目的 MAC 等于：48'h111111111111，输入端口号为 8'b00000001(0 号端口)，得到的输出端口号为 8'b00000010(1 号端口)；

当输入 TCP_B 报文时，此时目的 MAC 等于：48'h222222222222，输入端口号为 8'b00000010(1 号端口)，得到的输出端口号为 8'b00000001(0 号端口)；

因此能够得到结论：TCP_A 是 0 号端口进，1 号端口出；TCP_B 是 1 号端口进，0 号端口出。二层交换的功能能够正常运行。



MI 模块信号定义表

信号名称	方向	信号描述
i_smac_valid	i	MI 接收源 mac 有效指示
iv_smac[47:0]	i	MI 接收源 mac 值
iv_sport[7:0]	i	MI 接收源端口
o_smac_ready	o	MI 输出 mac 接收状态指示，置 1 时有效
o_smac_valid	o	MI 输出源 mac 有效指示信号
o_smac	o	MI 判别需查询端口号的源 mac
iv_s_index_valid	i	MI 查询源 mac 得到端口号有效指示信息
iv_s_index	i	MI 查询源 mac 得到的端口号信息
o_ctrl_mac_wr	o	MI 输出配置转发表项 mac 使能
ov_ctrl_mac[47:0]	o	MI 输出配置转发表项 mac 值
ov_ctrl_index[7:0]	o	MI 输出配置转发表项端口值

5.2.2 模块处理流程

模块复位后，当检测到 i_smac_valid 为高，将 iv_smac[47:0]和 iv_sport[7:0]一起写入 fifo 中，当 fifo 不为空的时候，启动状态机 rv_state_s，开始进行对源 mac（iv_smac）和源端口（iv_sport）的处理。

- smac and sport process：此逻辑块的功能是处理已缓存到 fifo 中的目的 smac 和对应端口号 sport，需执行以下功能
 - 1.读出 fifo 内缓存 smac 输出给 LUP 模块。
 - 2.等待 smac 查找(LUP)模块的有效查找结果索引端口值(iv_s_index)
 - 3.对当前索引端口值进行判别，如果索引端口值（iv_s_index）与当前输入的源端口号 sport 一致，则无需重新配置查表模块，若当前 mac 查找的索引端口值（iv_s_index）与源端口号不一致，则需要更新配置当前 mac 表项。
 - 4.根据判别结果，输出配置信息（ov_ctrl_mac）。
- smac fifo：此逻辑块的例化了一个 fifo 用于缓存从接收端口得到的源 mac。

5.2.3 仿真时序分析

仿真模拟以下情况：

对 0 号端口发送 64 字节的 TCP_A 报文，对 1 号端口发送 64 字

```
TCP_A[511:0] <= {128'h1111111111111111222222222222222208004500,
                  128'h00320000400040063ac4000000010000,
                  128'h0002000000000000000000000000000000000000,
                  128'h0000ffd800000000000000000000000000000000};
TCP_A[511:0] <= {128'h222222222222111111111111111108004500,
                  128'h00320000400040063ac4000000020000,
                  128'h0001000000000000000000000000000000000000,
                  128'h0000ffd800000000000000000000000000000000};
```

当输入 TCP_A 报文时,此时源 MAC 等于: 48'h222222222222, 输入端口号为 8'b00000001(0 号端口),存入 LOOKUP 中 MAC 地址与端口的对应关系为:

当输入 TCP_A 报文时，此时源 MAC 等于：48' h111111111111，输入端口号为 8' b00000001(0 号端口)，存入 LOOKUP 中 MAC 地址与端口的对应关系为：

[illegible]

图 5-3 TCP A 报文发送时的 manager index 仿真图

LUP 模块信号定义表

信号名称	方向	信号描述
i_dmac_valid	i	LUP 接收目的 mac 有效指示
iv_dmac[47:0]	i	LUP 接收目的 mac 值
ov_d_index_valid[15:0]	o	LUP 输出目的 mac 索引有效指示信号
ov_d_index[127:0]	o	LUP 输出目的 mac 索引值
i_smac_valid	i	LUP 接收源 mac 有效指示
iv_smac[47:0]	i	LUP 接收源 mac 值
iv_smac_port[7:0]	i	LUP 接收源端口值
ov_s_index_valid[15:0]	o	LUP 输出源索引有效指示信号
ov_s_index[127:0]	o	LUP 输出源索引值
i_ctrl_mac_wr	i	LUP 输入配置 mac 使能指示
i_ctrl_mac[47:0]	i	LUP 输入配置 mac 值
i_ctrl_index[7:0]	i	LUP 输入配置端口值
LOCAL BUS 总线		
i_ctrl_cs_n	i	Local bus 总线输入片选信号
i_ctrl_cmd	i	Local bus 总线操作指令信号
iv_ctrl_addr[15:0]	i	Local bus 总线输入地址
iv_ctrl_datain[31:0]	i	Local bus 总线输入数据
o_ctrl_ack_n	o	Local bus 总线输出应答
ov_ctrl_dataout[31:0]	o	Local bus 总线输出数据

5.3.2 模块处理流程

模块复位后，当检测到 i_smac_valid 为高，将 iv_smac[47:0]和 iv_sport[7:0]一起写入 fifo 中，当 fifo 不为空的时候，启动状态机 rv_state_s，开始进行对源 mac（iv_smac）和源端口（iv_sport）的处理。

- local bus: 此逻辑块的功能是通过 local bus 端口将当前的 mac 和表项中对应的端口号传递给上位机进行表项信息打印；
- cfg entry: 此逻辑块将当前需要配置的表项信息 mac 和端口号进行更新；
- entry_blk: 对输入的 mac 进行端口索引值输出，启动表项配置
- time_count: 表项更新时间计数器

六、部分代码注释

```

125 WRITE_C:begin //local bus 总线写之后跳转应答状态
126     cfg_state <= ACK_C;
127 end
128 READ_C:begin
129     cfg_state <= WAIT_C; //local bus总线读状态之后等一拍得到响应，跳转应答状态
130 end
131 WAIT_C:begin
132     cfg_state <= ACK_C; //等待响应
133 end
134 ACK_C:begin
135     case(iv_ctrl_addr[7:2])
136         6'h0: ov_ctrl_dataout <= rv_entry_valid[iv_ctrl_addr[11:8]]; //读取地址0，打印当前转发发表有效表项地址，取值0-15
137         6'h1: ov_ctrl_dataout <= rv_entry[iv_ctrl_addr[11:8]][47:32]; //读取地址1，打印转发表mac地址高16位
138         6'h2: ov_ctrl_dataout <= rv_entry[iv_ctrl_addr[11:8]][31:0]; //读取地址2，打印转发表mac地址低32位
139         6'h3: ov_ctrl_dataout <= rv_index[iv_ctrl_addr[11:8]]; //读取地址3，打印端口号
140         6'h3f: ov_ctrl_dataout <= rv_age_time; //读取地址3f，打印老化时间
141     endcase
142     if(ctrl_cs) begin
143         o_ctrl_ack_n <= 1'b0;
144         cfg_state <= ACK_C; //当ctrl_cs一直为高时，保持响应状态持续到case语句执行所有信息打印完
145     end
146     else begin
147         o_ctrl_ack_n <= 1'b1;
148         cfg_state <= IDLE_C;
149     end
150 end

```

module lookup/local bus/cfg_state

```

) //*****
) //cfg entry
) //*****
) always@(posedge i_clk or negedge i_rst_n)
)   if(!i_rst_n) begin
)       rv_ctrl_mac_wr <= 16'b0;
)       rv_ctrl_mac <= 48'b0;
)       rv_ctrl_index <= 8'b0;
)   end
)   else begin
)       if(i_ctrl_mac_wr) begin
)           casez(rv_entry_valid) //初始状态rv_entry_valid[0]=0，当配置表项启动i_ctrl_mac_wr=1，
)               16'b????????????0: rv_ctrl_mac_wr[0] <= 1'b1; //优先命中第一条表项，rv_ctrl_mac_wr[0]置1，写入entry_blk[0] module
)               16'b????????????01: rv_ctrl_mac_wr[1] <= 1'b1; //第一条写入后，rv_entry_valid[0]置1，当再次启动配置表项i_ctrl_mac_wr=1，写入entry_blk[1]
)               16'b????????????011: rv_ctrl_mac_wr[2] <= 1'b1; //该case语句就会依次向下一条表项写，同时rv_entry_valid类似bitmap，当有信息写入对应的表项即该位置1
)               16'b????????????0111: rv_ctrl_mac_wr[3] <= 1'b1; //当表项老化时间x_age_flag达到，该表项内容无效，对应的rv_entry_valid[n]会清0
)               16'b????????????01111: rv_ctrl_mac_wr[4] <= 1'b1;
)               16'b????????????011111: rv_ctrl_mac_wr[5] <= 1'b1;
)               16'b????????????0111111: rv_ctrl_mac_wr[6] <= 1'b1;
)               16'b????????????01111111: rv_ctrl_mac_wr[7] <= 1'b1;
)               16'b????????????011111111: rv_ctrl_mac_wr[8] <= 1'b1;
)               16'b????????????0111111111: rv_ctrl_mac_wr[9] <= 1'b1;
)               16'b????????????01111111111: rv_ctrl_mac_wr[10] <= 1'b1;
)               16'b????????????011111111111: rv_ctrl_mac_wr[11] <= 1'b1;
)               16'b????????????0111111111111: rv_ctrl_mac_wr[12] <= 1'b1;
)               16'b????????????01111111111111: rv_ctrl_mac_wr[13] <= 1'b1;
)               16'b????????????011111111111111: rv_ctrl_mac_wr[14] <= 1'b1;
)               16'b01111111111111111111: rv_ctrl_mac_wr[15] <= 1'b1;
)               default: rv_ctrl_mac_wr <= 16'b0;
)           endcase
)           rv_ctrl_mac <= iv_ctrl_mac ;
)           rv_ctrl_index <= iv_ctrl_index ;
)       end
)       else begin
)           rv_ctrl_mac_wr <= 16'b0;
)           rv_ctrl_mac <= 48'b0;
)           rv_ctrl_index <= 8'b0;
)       end
)   end
) end

```

module lookup/cfg entry

```

| *****
| //          ov_entry Config
| *****
| always @(posedge i_clk or negedge i_rst_n)
|     if(i_rst_n == 1'b0) begin
|         ov_entry      <= 48'b0;
|         o_entry_valid <= 1'b0;
|         ov_index      <= 8'b0;
|     end
|     else begin
|         if(i_ctrl_mac_wr) begin //与module lookup对应的cfg_entry部分联系看，当启动配置i_ctrl_mac_wr=1，
|             ov_entry      <= iv_ctrl_mac; //配置mac赋值给表项，同时将该表项当前mac输出
|             o_entry_valid <= 1'b1; //表项有值指示信号置1
|             ov_index      <= iv_ctrl_index; //配置端口号赋值给表项，同时将该表项当前端口号输出
|         end
|         else if(r_age_flag) begin //老化时间到，该表项对应值清空
|             ov_entry      <= 48'b0;
|             o_entry_valid <= 1'b0;
|             ov_index      <= 8'b0;
|         end
|         else begin
|             ov_entry      <= ov_entry      ; //其它状态，该表项值保持
|             o_entry_valid <= o_entry_valid ;
|             ov_index      <= ov_index      ;
|         end
|     end
| end

```

module entry_blk/ov_entry Config

```

85 :
86 ⊖ *****
87 //          ov_entry Lookup
88 ⊖ *****
89 :
90 ⊖ always @(posedge i_clk or negedge i_rst_n) //源mac地址查表项，命中o_s_hit置1，输出端口号ov_s_index，
91 ⊖     if(i_rst_n == 1'b0) begin //没有命中，o_s_hit置0，端口号为0
92 :         o_s_hit <= 1'b0;
93 :         ov_s_index <= 8'b0;
94 ⊖     end
95 ⊖     else begin
96 :         if(i_smac_valid == 1'b1) begin
97 :             if((ov_entry == iv_smac)&(o_entry_valid == 1'b1)) begin
98 :                 o_s_hit <= 1'b1;
99 :                 ov_s_index <= ov_index;
100 ⊖             end
101 ⊖             else begin
102 :                 o_s_hit <= 1'b0;
103 :                 ov_s_index <= 8'b0;
104 ⊖             end
105 ⊖         end
106 ⊖         else begin
107 :             o_s_hit <= 1'b0;
108 :             ov_s_index <= 8'b0;
109 ⊖         end
110 ⊖     end
111 :
112 :
113 ⊖ always @(posedge i_clk or negedge i_rst_n) //目的mac地址查表项，命中o_d_hit置1，输出端口号ov_d_index
114 ⊖     if(i_rst_n == 1'b0) begin //没有命中，o_d_hit置0，端口号为0
115 :         o_d_hit <= 1'b0;
116 :         ov_d_index <= 8'b0;
117 :

```

module entry_blk/ov_entry Lookup


```

always @(posedge i_clk or negedge i_rst_n)
    if(i_rst_n == 1'b0) begin
        r_age_flag <= 1'b0; //老化标志位，高电平有效，有效时会将表项中的值清空
        rv_cnt_second <= 0'd0; //老化计数器，单位为s
    end
    else begin
        if(o_entry_valid) begin //只有当表项中存在值时，才会开始计数
            if(r_age_reset_flag) //老化计数器复位标志位，高电平有效，有效时会将老化计数器清零
                rv_cnt_second <= 0;
            else if(i_second_flag) //时钟脉冲标志位，高电平有效，每当time_count模块计数时间满1s时，该信号会置高，会使老化计数器加1
                rv_cnt_second <= rv_cnt_second + 1;
            else
                rv_cnt_second <= rv_cnt_second;
        end
        else begin
            rv_cnt_second <= 0;
        end

        if(rv_cnt_second >= iv_age_time) //当老化计数器大于等于老化时间时，会使老化标志位置高
            r_age_flag <= 1'b1;
        else
            r_age_flag <= 1'b0;
    end
end

always @(posedge i_clk or negedge i_rst_n)
    if(i_rst_n == 1'b0)
        r_age_reset_flag <= 1'b0; //老化计数器复位标志位
    else
        if(ov_entry == iv_smac && o_entry_valid && ov_index == iv_smac_port && i_smac_valid) //当同时满足：1. 表项存储的值等于源MAC的值。2. 表项有效位为高
            r_age_reset_flag <= 1'b1; //3. 索引值等于源MAC输入端口。4. 源MAC有效位为高。 会将老化计数器复位标志位拉高。
        else
            r_age_reset_flag <= 1'b0;
    end

```

module entry_blk/old time

七、功能验证

首先，本实验需要用户掌握以太网交换机原理，通过详细阅读本文档 5.1.2 节和 5.2.2 节两个块处理流程，用 verilog 代码实现 5.1.2 及 5.2.2 所描述的以下几点功能：

- **dmac and dport process**：此逻辑块的功能是处理已缓存到 fifo 中的目的 dmac 和对应端口号，需执行以下功能
 1. 读出 fifo 内缓存 dmac，根据 dmac 判断是广播 mac 还是单播，将需要进行目的 mac 端口查找的单播 mac 输出。
 2. 若是广播 dmac，则将目的端口号置为泛洪输出（按位取反输出），若是单播 mac，则将目的 o_dmac 输出到查找（LUP）模块，对目的 mac 对应的端口号表项进行查询。
 3. 等待查找（LUP）模块输入有效的目的端口索引号(iv_d_index)。
 4. 处理查找（LUP）模块查询的目的端口索引号(iv_d_index)，这里因为每个表项储存一个端口号（8b）信息，一共设置了 16 条表项，所以一共是 128b，对这 128b 信息进一步处理，得到 8b 的端口信息（ov_outport）。
 5. 将最终处理得到的端口信息（ov_outport）输出。
- **smac and sport process**：此逻辑块的功能是处理已缓存到 fifo 中的目的 smac 和对应端口号 sport，需执行以下功能
 1. 读出 fifo 内缓存 smac 输出给 LUP 模块。
 2. 等待 smac 查找(LUP)模块的有效查找结果索引端口值(iv_s_index)
 3. 对当前索引端口值进行判别，如果索引端口值（iv_s_index）与当前输入的源端口号 sport 一致，则无需重新配置查表模块，若当前 mac 查找的索引端口值（iv_s_index）与源端口号不一致，则需要更新配置当前 mac 表项。
 4. 根据判别结果，输出配置信息（ov_ctrl_mac）。

通过补全以上两个块功能，完整的实现二层交换中源 mac 地址对转发表项建立更新，目的 mac 查询转发端口的主要功能。

其次，参考“[vivado 安装使用](#)”或自学等方式学会 vivado 平台工程仿真，综合，编译，实现等步骤并生成 bit 流文件，参考“[boot.bin 文件生成\(vitis\)](#)”生成 boot.bin 文件。

最后，按照“[硬件二层交换实验操作手册](#)”描述步骤操作，顺利打印出接入 openbox 平台的设备的 mac 地址。打印结果如下图所示：

```
当前硬件老化时间 :1e
ID      PORT0      PORT1      PORT2      PORT3      PORT4      PORT5      PORT6      PORT7
0        .          .          .          .          .          .          .          .
1        .          .          .          .          .          .          .          .
当前硬件老化时间 :1e
ID      PORT0      PORT1      PORT2      PORT3      PORT4      PORT5      PORT6      PORT7
0        .          .          .          .          .          .          .          .
1        .          .          .          .          .          .          .          .
当前硬件老化时间 :1e
ID      PORT0      PORT1      PORT2      PORT3      PORT4      PORT5      PORT6      PORT7
0        .          .          .          .          .          .          .          .
1        .          .          .          .          .          .          .          .
当前硬件老化时间 :1e
ID      PORT0      PORT1      PORT2      PORT3      PORT4      PORT5      PORT6      PORT7
0        .          .          .          .          .          .          .          .
1        .          .          .          .          .          .          .          .
```